Window Functions

ROW_NUMBER() and a Relevant MySQL Syntax

- One of the syntax types available for applying window functions:

SQL

```
SELECT
    ...,
    ROW_NUMBER() OVER () AS ...
FROM
    ...
;
```

MySQL window function operations can only be accessed through the OVER clause, so essentially if we don't use an OVER clause, we won't have a window function. This is because the OVER clause relates to the window over which the function evaluation will occur. In the example, this means the OVER clause will lead to a window specification. There, we will define how we want to organise the rows where the row number function will be providing rank values, complying with the syntax we've chosen to use.

```
USE employees;

SELECT
    emp_no,
    salary,
    ROW_NUMBER() OVER (PARTITION BY emp_no) AS row_num
FROM
    salaries;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

emp_no	salary	row_num
10001	60117	1
10001	62102	2
10001	66074	3
10001	66596	4
10001	66961	5
10001	71046	6
10001	74333	7
10001	75286	8

Result 6

The clause we will encounter most often is PARTITION BY. PARTITION BY will organise the data from the salaries table into groups, or, as we just clarified, partitions. To elaborate on that, we can also say that partition by employee number will reorganise the data into partitions. All records containing the same employee number will be considered as one partition. That's why, as we execute this query, we see row numbers 1, 2, 3, etc. up to 17, appearing for the records related to employee number 10001.

```
USE employees;

SELECT
    emp_no,
    salary,
    ROW_NUMBER() OVER (PARTITION BY emp_no ORDER BY) AS row_num
FROM
    salaries;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch ro

emp_no	salary	row_num
10001	76884	10
10001	80013	11
10001	81025	12
10001	81097	13
10001	84917	14
10001	85112	15
10001	85097	16
10001	88958	17

Lecturer notes: It will make more sense to rank the highest salary value as 1, the second-highest as 2, and so on. Therefore, what can we do to arrange the data and the partitions by salary value in descending order? In this case, we see how intuitive the MySQL syntax can be. It allows us to add an ORDER BY clause within the window specification to obtain the desired reorganisation of the salary values. To do that, we must add the following bit of code after the employee number. ORDER BY: salary.

```
USE employees;

SELECT
    emp_no,
    salary,
    ROW_NUMBER() OVER (PARTITION BY emp_no ORDER BY salary DESC) AS row_num
FROM
    salaries;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

emp_no	salary	row_num
10001	88958	1
10001	85112	2
10001	85097	3
10001	84917	4
10001	81097	5
10001	81025	6
10001	80013	7
10001	76884	8

Result 9

Finally, we need to remember that the PARTITION BY clause is not mandatory. Should we choose not to use it in our query, we can still take advantage of other clauses. For instance, we can employ an OVER and an ORDER BY clause in the following way to arrange the salary values. In an ascending or descending order, and obtain the corresponding ranked values.

```
USE employees;

SELECT
    emp_no,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM
    salaries;
```

ROW_NUMBER() and a Relevant MySQL Syntax

window specification:	action:
None = an *empty* OVER clause	ROW_NUMBER() will perform the relevant evaluations on all query rows = a *single* partition
containing PARTITION BY	the data will be organized into partitions
containing ORDER BY	arrange the values (in an ascending or descending order)

The ROW_NUMBER() Ranking Window Function - Exercise #1

Retrieve everything stored in the employee number (`emp_no`) and department number (`dept_no`) columns from the department manager (`dept_manager`) table. Add a third column named `row_num` containing a row number, starting from 1 and incrementing for each row in the obtained output, ordered by `emp_no` in descending order.

This topic is covered in

Lecture 266: The ROW_NUMBER() Ranking Window Function and the Relevant MySQL Syntax

```
SELECT
emp_no,
dept_no,
row_number() OVER (ORDER BY emp_no DESC) AS row_num
FROM
dept_manager;
```

Run query Reset

Result Success

emp_no	dept_no	row_num
10008	d003	1
10007	d002	2
10006	d001	3

The ROW_NUMBER() Ranking Window Function - Exercise #2

Retrieve everything stored in the employee number (`emp_no`), first name (`first_name`), and last name (`last_name`) columns from the `employees` table. Add a fourth column named `row_num`, which partitions the data by last name, sorts it by employee number in ascending order, and assigns a row number starting from 1 and incrementing for each row in every partition.

This topic is covered in

Lecture 266: The ROW_NUMBER() Ranking Window Function and the Relevant MySQL Syntax

```
SELECT
emp_no,
first_name,
last_name,
row_number() OVER (PARTITION BY last_name ORDER BY emp_no ASC) AS row_num
FROM
employees;
```

Run query Reset

Result Success

10005	Kyoichi	Maliniak	1
10009	Sumant	Peac	1
10010	Duangkaew	Piveteau	1
10002	Bezalel	Simmel	1
10007	Tzvetan	Zielinski	1

A Note on Using Several Window Functions - Exercise #1

Retrieve the employee number (`emp_no`) and job title (`title`) from the `titles` table, and the salary (`salary`) from the `salaries` table. Add a column to the left, named `row_num1`, starting from 1 incrementing by 1 for each row from the obtained result. Also, add a fifth column, named `row_num2`, which provides a row position for each record per employee, starting from the total number of records obtained for that employee and continuing down to 1.

row_num1	emp_no	title	salary	row_num2

```sql
SELECT
row_number() OVER () AS row_num1,
t.emp_no,
t.title,
s.salary,
row_number() OVER (PARTITION BY t.emp_no ORDER BY s.salary DESC) AS row_num2
FROM titles t
JOIN salaries s
ON t.emp_no = s.emp_no
WHERE t.emp_no < 10007 AND t.title = "Staff"
ORDER BY s.salary, row_num1 ASC;
```

Run query Reset

Result Success

A Note on Using Several Window Functions - Exercise #2

Retrieve the employee number (`emp_no`) and job title (`title`) from the `titles` table, and the salary (`salary`) from the `salaries` table. Add a fourth column, named `row_num1`, containing starting from 1 incrementing by 1 for each row for every employee from the obtained result. Also, add a fifth column, named `row_num2`, which provides the opposite values - starting from the total number of records obtained for a given employee and continuing down to 1.

emp_no	title	salary	row_num1	row_num2
10002	Staff	65828	1	6
10002	Staff	65909	2	5
10002	Staff	67534	3	3

```sql
SELECT
t.emp_no,
t.title,
s.salary,
row_number() OVER (PARTITION BY t.emp_no ORDER BY salary ASC) AS row_num1,
row_number() OVER (PARTITION BY t.emp_no ORDER BY salary DESC) AS row_num2
FROM titles t
JOIN
salaries s
ON t.emp_no = s.emp_no
WHERE t.emp_no < 10007 and t.title = "Staff"
ORDER BY t.emp_no, s.salary, row_num1 ASC;
```

Run query Reset

Result Success

```sql
USE employees;

SELECT
    emp_no,
    salary,
    ROW_NUMBER() OVER w AS row_num
FROM
    salaries
WINDOW w AS (PARTITION BY emp_no ORDER BY salary DESC);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

emp_no	salary	row_num
10001	88958	1
10001	85112	2
10001	85097	3
10001	84917	4
10001	81097	5
10001	81025	6

Result 2

Output

Action Output

#	Time	Action	Message
1	20:33:57	USE employees	0 row(s) affected
2	20:37:07	SELECT emp_no, salary, ROW_NUMBER() OVER (PARTITION BY emp_no ...	967330 row(s) returned
3	20:38:21	SELECT emp_no, salary, ROW_NUMBER() OVER w AS row_num FROM sal...	967330 row(s) returned

Another way to write a Windows syntax – notice how we used the alias "w" in this string of code. (Ultimately redundant, but hey, who knows when it might come in handy)

MySQL Window Functions Syntax - Exercise #1

Similar to *"The ROW_NUMBER() Ranking Window Function - Exercise #2"*, retrieve everything stored in the employee number (`emp_no`), first name (`first_name`), and last name (`last_name`) columns from the `employees` table. Add a fourth column named `row_num`, which partitions the data by first name, sorts it by employee number in ascending order, and assigns a row number starting from 1 and incrementing for each row in every partition.

Use the WINDOW keyword to solve the exercise.

```
SELECT
e.emp_no,
e.first_name,
e.last_name,
row_number() OVER w AS row_num
from employees e
WINDOW w AS (PARTITION BY e.first_name ORDER BY e.emp_no ASC);
```

Run query Reset All changes saved | Line 7, C

Result Success